

BUJUMBURA INTERNATIONAL UNIVERSITY

FACULTE DE SCIENCE ET TECHNOLOGIE

Cours de Langage de compilation

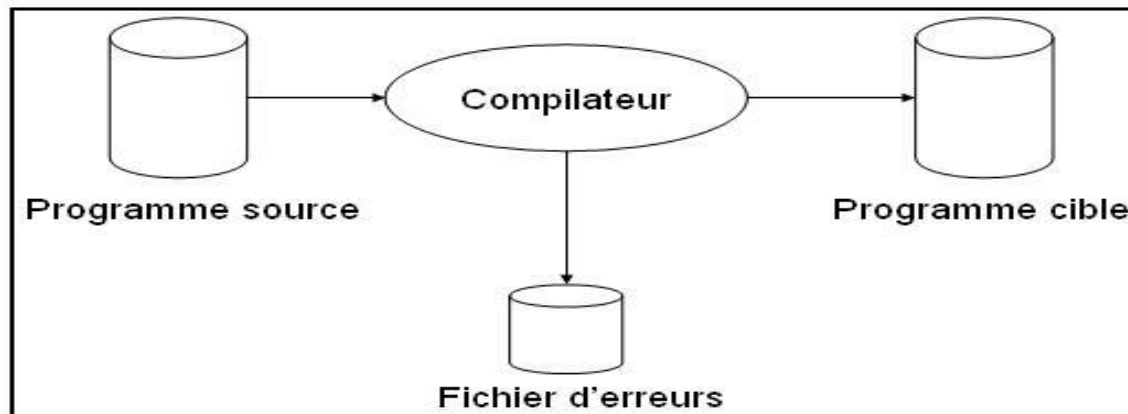
Titulaire: Pr. NIYONSABA Thérance

A/A 2025-2026

Introduction

Definitions

Un compilateur est un programme qui lit un programme (**programme source**) écrit dans un langage de haut-niveau (C++ ou Java,..) et le traduit en un programme équivalent écrit dans un autre langage (cible). Cette opération est effectuée en signalant toute présence d'erreur dans le programme source. A ne pas le confondre avec l'interpréteur.



Exemple:

- gcc GCC(GNU Compiler Collection) : compilateur de C/C++, vers de l'assembleur.
- javac : compilateur de Java vers du byte code.
- latex2html : compilateur de Latex vers du Postscript ou du HTML.

Classes de langages de programmation

On distingue:

- langages de bas niveau (le langage machine et le langage assembleur);
- le langage de haut niveau et le langage de quatrième génération;

1. Le langage machine

Il est écrit en binaire, une suite de **0 et 1**; C'est le langage de programmation le plus simple à comprendre pour l'ordinateur. le langage machine est un langage qui est très difficile à comprendre pour un programmeur.

Exemple:0001:chargement ; 0010: opération d'addition

2 Le langage assembleur (langage de deuxième génération)

Le langage de programmation qui se rapproche le plus du langage machine. L'instruction MOV remplace la plupart des instructions binaire du processeur.

Le langage de haut niveau

C'est un niveau langage de programmation qui se rapproche beaucoup plus d'un langage dit **naturel** que du langage machine.

Le code d'un programme fait avec un langage de haut niveau utilise des mots issus d'un langage plus familier pour l'être humain en faisant abstraction des caractéristiques techniques du matériel.

Ils ont un niveau d'abstraction beaucoup plus élevé.

Le langage beaucoup plus compréhensible, rapide, et facile d'utilisation pour un programmeur.

Le programme peut donc facilement être fonctionnel sur plusieurs ordinateurs.

Néanmoins, il est lent car, il faut traduire le code du programme en langage machine avant de l'exécuter «compilation». Exemple: C++, Java,...

Le langage de quatrième génération

Un langage qui se rapproche encore plus du langage naturel que le langage de troisième génération;

Le langage de quatrième génération est un langage qui est conçu pour résoudre des problèmes spécifiques (par ex. accès aux bases de données et faire des requêtes, création des sites,...);

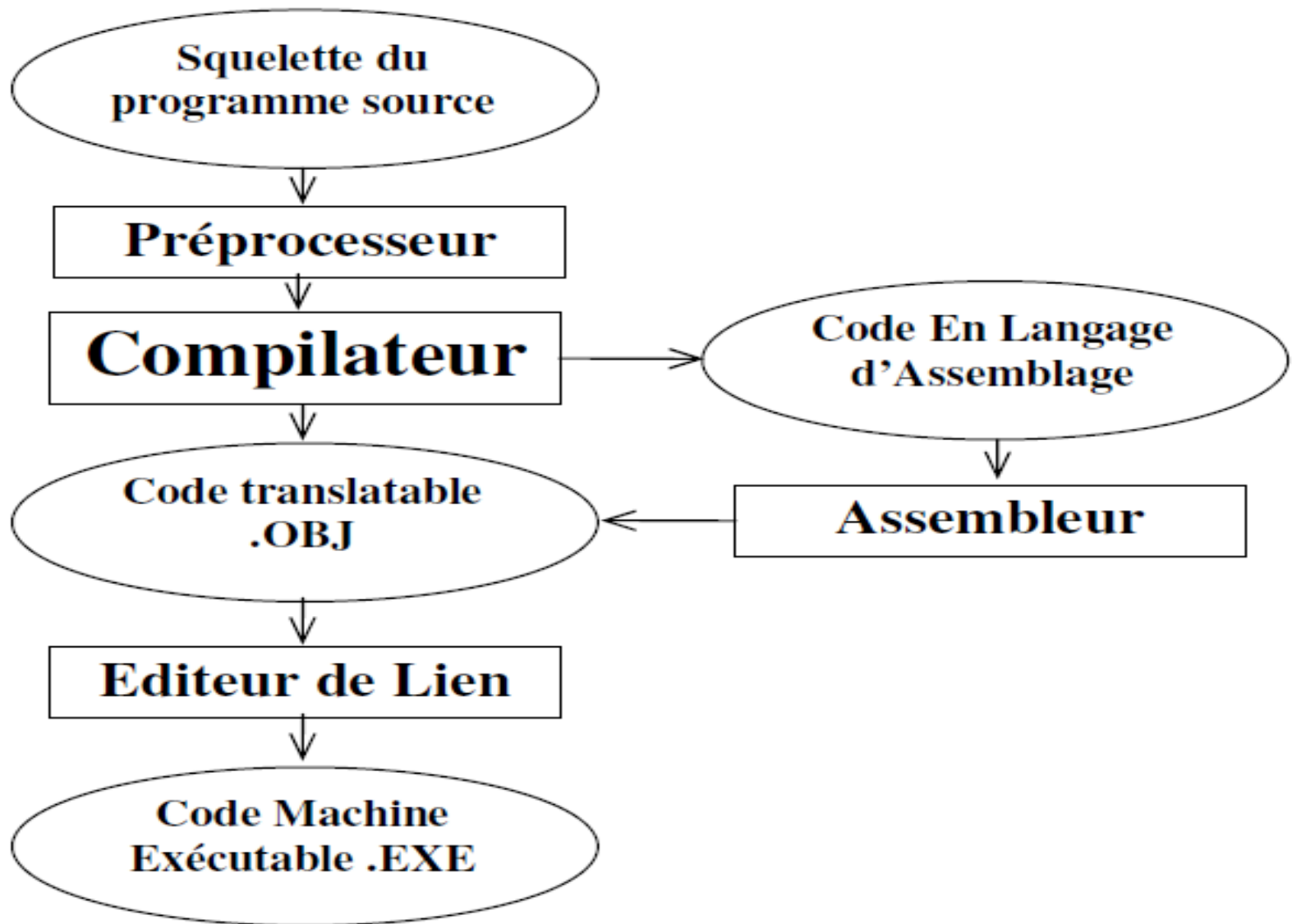
Ces langages permettent au programmeur de programmer une application spécifique à un sujet et de le faire avec une grande lisibilité et très peu de code.

Avantages du compilateur

L'utilisation d'un compilateur permet de gagner plusieurs avantages :

- Il permet de programmer avec un niveau supérieur d'abstraction plutôt qu'en assembleur ou en langage machine.
- Il permet de faciliter la tâche de la programmation en permettant d'écrire des programmes lisibles, modulaires, maintenables et réutilisables.
- Il permet de s'éloigner de l'architecture de la machine et donc d'écrire des programmes portables.
- Il permet de détecter des erreurs et donc d'écrire des programmes plus fiables.

Schéma et environnement d'un compilateur



Le préprocesseur:

Un programme source peut être divisé en modules sur des fichiers séparés. La tâche de reconstitution du programme source est déléguée à un outil ne faisant pas partie du compilateur : **le préprocesseur**.

Ainsi, en langage C par exemple les instructions du préprocesseur sont présentées par le symbole # (#include , #define ...) et doivent être transformées avant que le compilateur démarre son traitement.

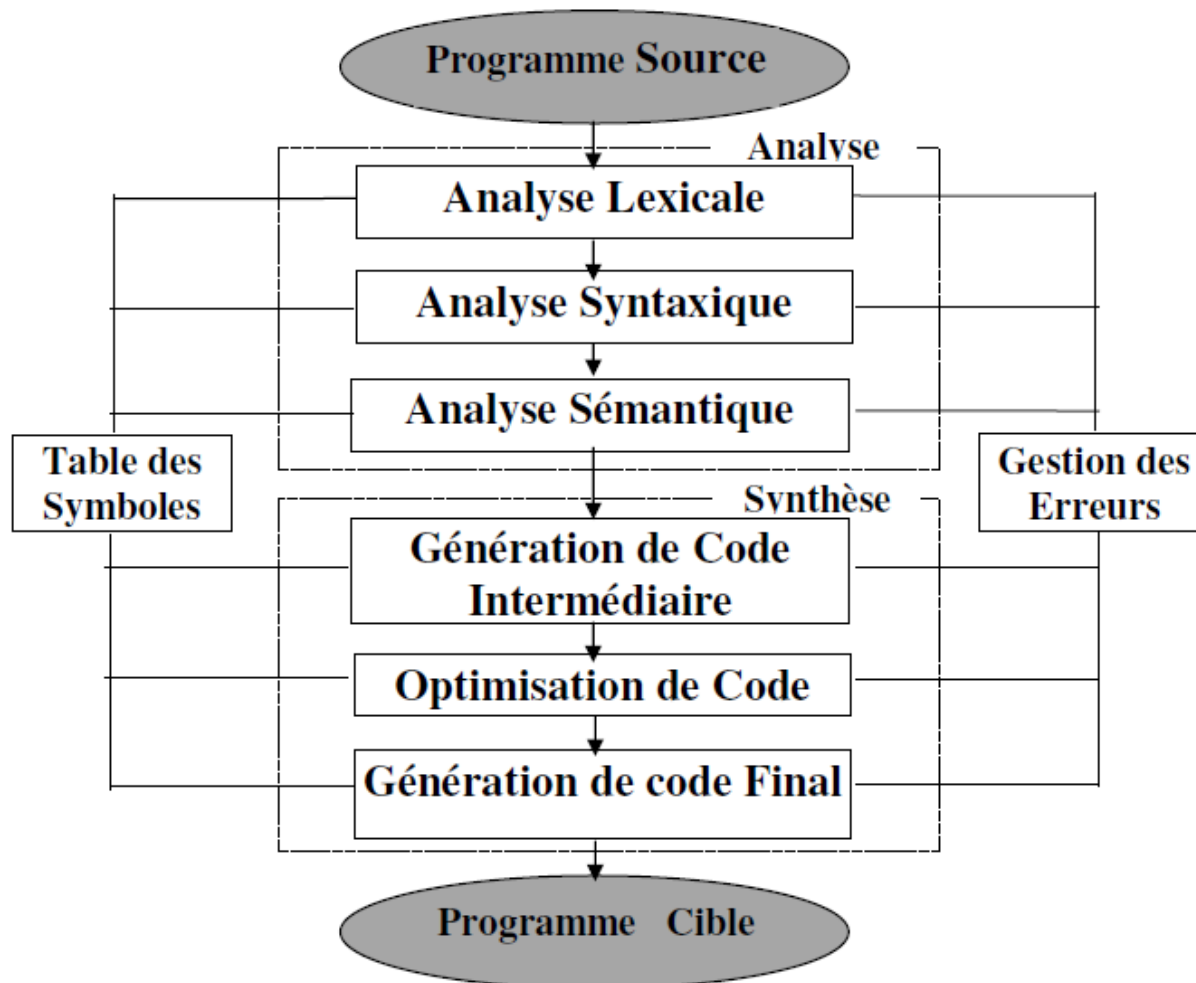
Editeur de lien (chargeur-relieur):

Après la phase de compilation on obtient du code en langage machine translatable (.obj). Ce code nécessite un traitement supplémentaire avant d'être exécutable. Il s'agit de l'édition de lien (Linkage) qui est confiée à l'éditeur de liens.

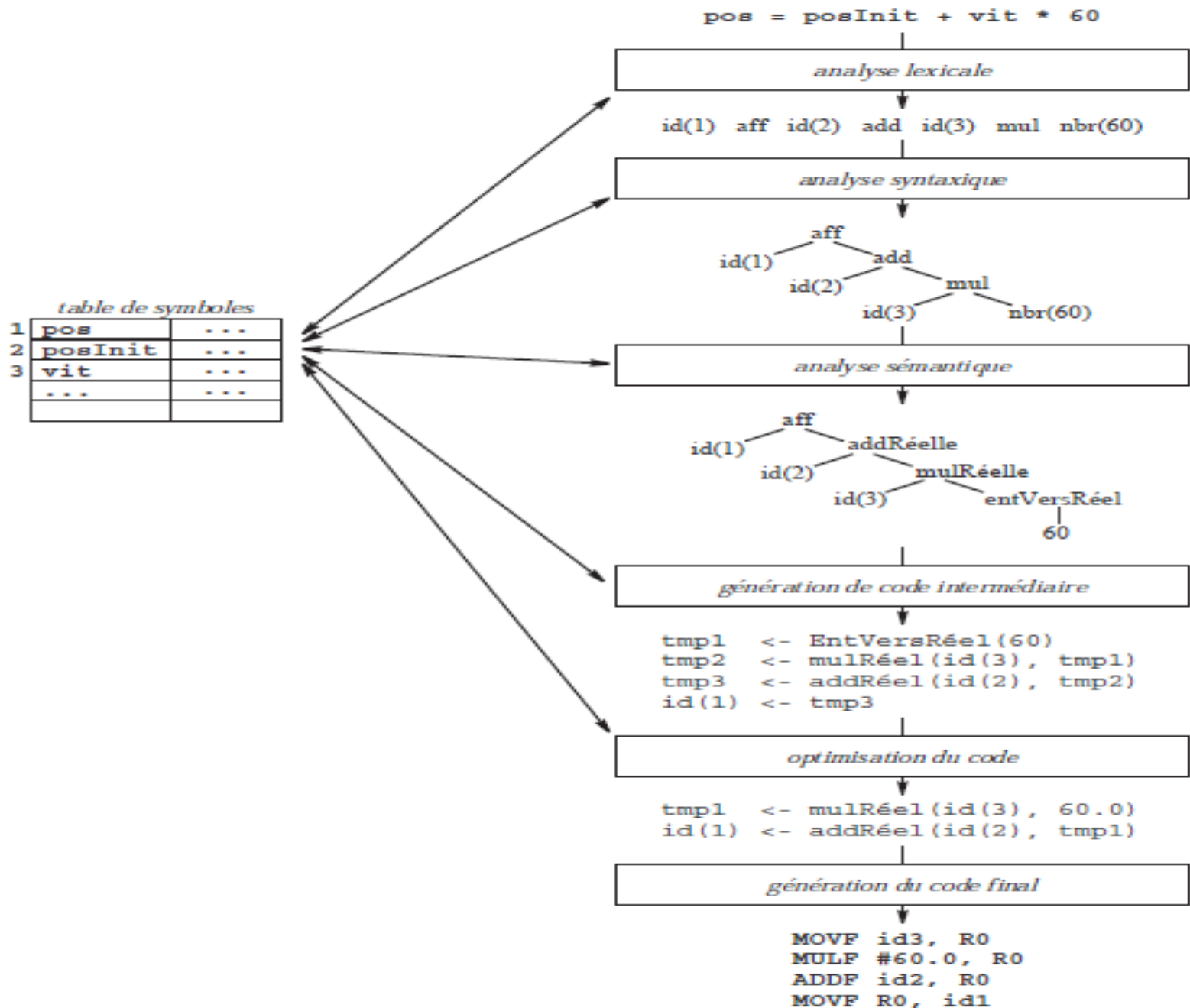
Chapitre 1. Phases de la compilation

Une phase est une opération transformant le programme d'une représentation à une autre.

Un compilateur typique peut être décomposé en six phases représentées dans les schémas ci-après.



Exemple



Chapitre 2 Analyse lexicale

L'analyse lexicale constitue la première phase d'un compilateur. Elle est faite **par l'analyseur lexical(scanner)** dont la matière première est l'ensemble des caractères du texte à compiler et dont la sortie est une séquence **d'unités lexicales ou jetons**(une classe de lexèmes de même catégorie) que l'analyseur syntaxique(parser) récupère ensuite. Les unités lexicales produites et plus précisément les identificateurs seront stockées dans une table appelée table des symboles

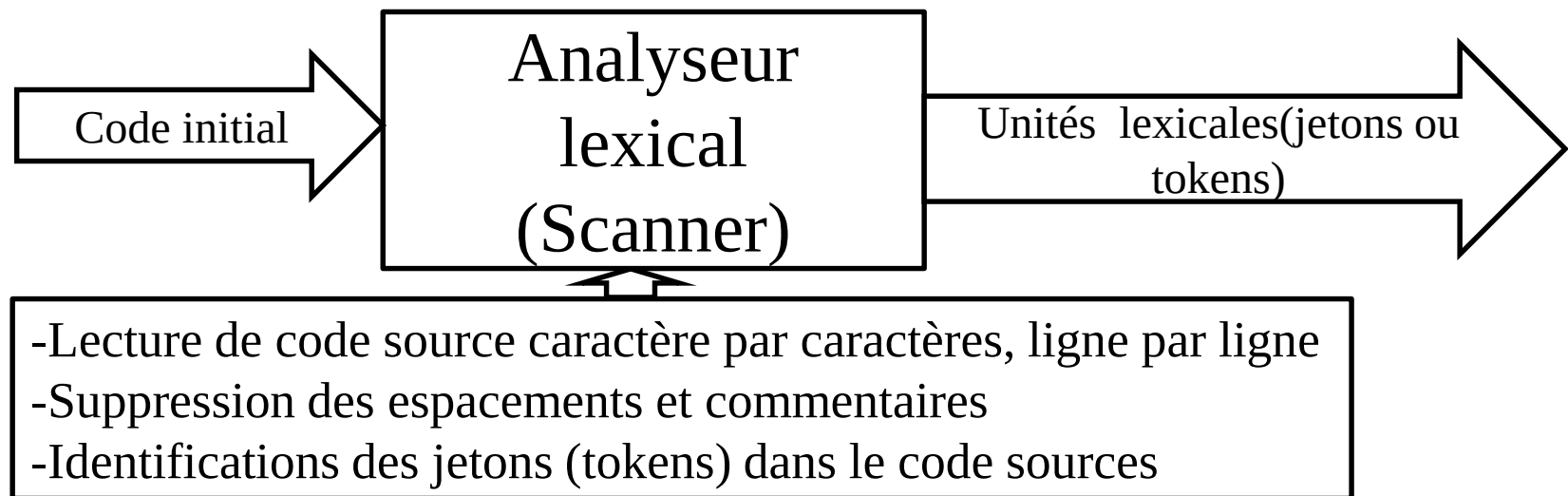


Table des symboles

On définit alors une unité lexicale comme un symbole qui représente une classe d'entités tirées à partir du fichier source et obéissant à une même règle. L'unité lexicale est donc un symbole terminal qui rentre dans la constitution du vocabulaire du langage source. Les unités lexicales produites sont stockées dans une table appelée **table des symboles.**

Un lexème est une suite de caractères du programme source qui constitue une entité élémentaire représentant une unité lexicale donnée.

Les principales unités lexicales sont:

Identificateurs (Identifiants): a,b,....

Mots clés: main, int, printf,while,if,...

Opérateurs(relations): =,==,+,. /

Symboles spéciaux:(),{};

Constantes:10,1,5,...

Table des symboles

Exemple1: a=b+c+10;

Identifiants: a, b, c

Operateurs : =,+

Constantes: 10

Exemple 2: int max(x, y);

Printf(“bonjour”);

Printf(“%i ”,&x);

Exemple de table de symbole:

Lex est un outil pour la construction automatique d'un analyseur lexical

Quelques erreurs détectées au niveau de l'analyse lexical

- Un caractère n'appartenant pas à l'alphabet du langage,
- Un commentaire non fermé,
- Une chaîne de caractère non terminée
- Une constante caractère contenant plus que 1 ou 2 caractères.
- Les caractères ASCII

Expressions régulières

Avant donc de commencer l'analyse d'un fichier texte, il est nécessaire de préciser quelles sont les unités lexicales qu'on peut accepter. La meilleure représentation utilisée est celle des **expressions régulières**.

Une expression régulière est une notation utilisée pour la description d'une unité lexicale d'un langage. Toutes les unités d'un langage dit régulier sont dénotées par des expressions régulières.

Expressions régulières:

1. Alphabet: C'est un ensemble fini et non vide d'éléments appelés des symboles ou caractères.

$A = \{ 0, 1 \}$ Alphabet binaire

$B = \{ a, b, c, !, \% \}$ est aussi un alphabet

2. Chaîne: C'est une séquence finie de symboles tirés d'un alphabet donné pour constituer une seule entité.

Soit l'alphabet $A = \{ a, b, c, d, e \}$,

e, a, abc, bbe, dd sont des chaînes de l'alphabet A .

3. Opérateurs de base:

- Opérateur de concaténation : « . »: La concaténation de deux chaînes est une chaîne formée par les symboles de la première chaîne suivie de ceux de la deuxième
- Opérateur d'Union : « | »: L'union « | » désigne que l'une ou l'autre de deux chaînes.
- Opérateur de Fermeture : « * »: L'opérateur de fermeture * désigne la concaténation d'une chaîne avec elle même un nombre quelconque de fois (éventuellement nul $\Rightarrow \varepsilon$).

Exemple 1: $(ab)^*$ signifie : $\varepsilon \mid ab \mid abab \mid ababab \dots$

Exemple 2: $(xy)^3$ signifie: $\varepsilon \mid xy \mid xyxy \mid xyxyxy$

- Opérateur de Fermeture positive : « + »: L'opérateur de fermeture positive + est utilisé pour designer la concaténation d'une chaîne avec elle même, une, 2 ou plusieurs fois.

$(S)^+$ notée aussi S^+ signifie : $S \mid S S \mid S S S \dots$

S^+ est équivalente à : $S S^*$

- Opérateur d'option : « ? » : l'opérateur d'option ? est utilisé pour désigner la chaîne sur laquelle il est appliqué ε .

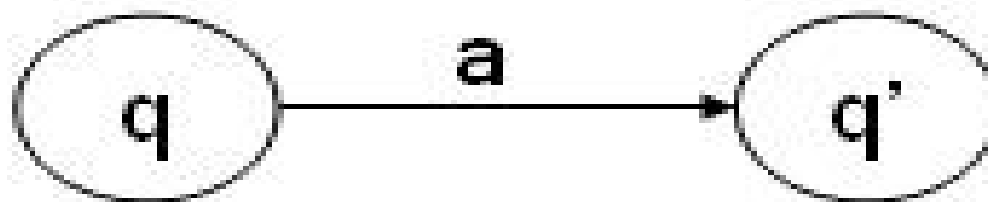
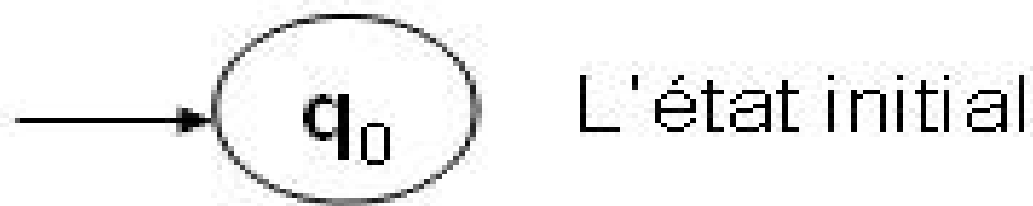
$(S)?$ Notée aussi $S?$ Signifie : $S \mid \varepsilon$

Les Automates à états Finis Déterministe (AFD)

Un Automate Fini est un diagramme graphique permettant de préciser schématiquement les étapes qu'il faut suivre pour identifier et extraire des lexèmes dont l'expression régulière est bien définie.

Il s'agit d'un ensemble d'états qui indique la progression d'un analyseur lexical.

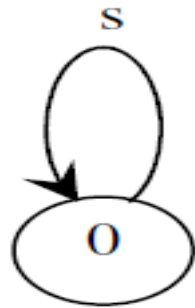
- Les états sont les sommets du graphe et sont représentés par des petits circles portant des numéros (ou des noms) qui les identifient. Cependant, ces numéros ou noms n'ont aucune signification opérationnelle.
- Un **état final** est représenté par deux cercles concentriques.
- L'état **initial** q_0 est marqué par une flèche qui pointe sur son cercle.
- Les transitions représentent les arcs du graphe. Une transition est dénotée par une flèche reliant deux états. Cette flèche porte une étiquette qui est un symbole de l'alphabet.



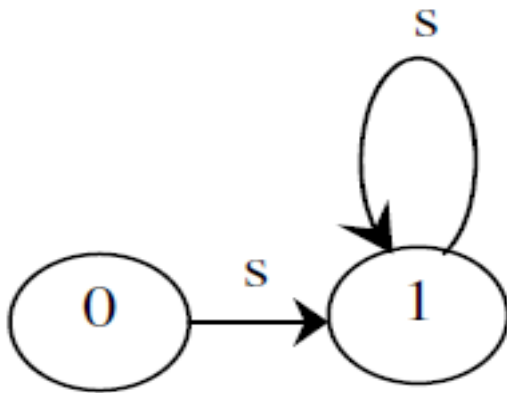
Une transition de l'état q à l'état q' avec le symbole d'entrée a : $\delta(q, a) = q'$

Un AFD est représenté par un graphe orienté étiquette

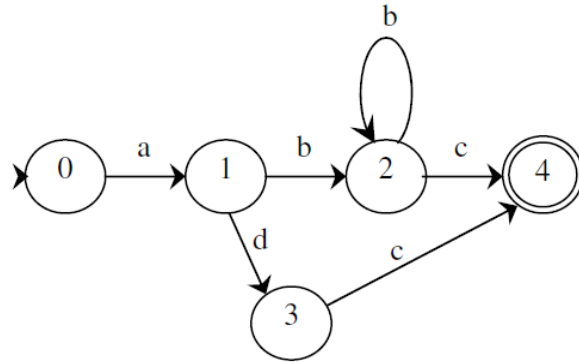
Si l'ER est une fermeture (s^*)



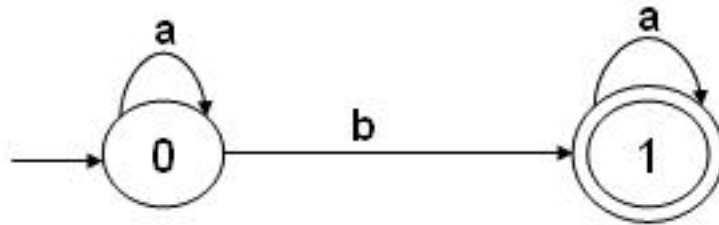
Si l'ER est une fermeture positive (s^+) :



Exemple 1: Pour l'expression régulière suivante construire l'automate correspondant: $a(b^+|d)c$

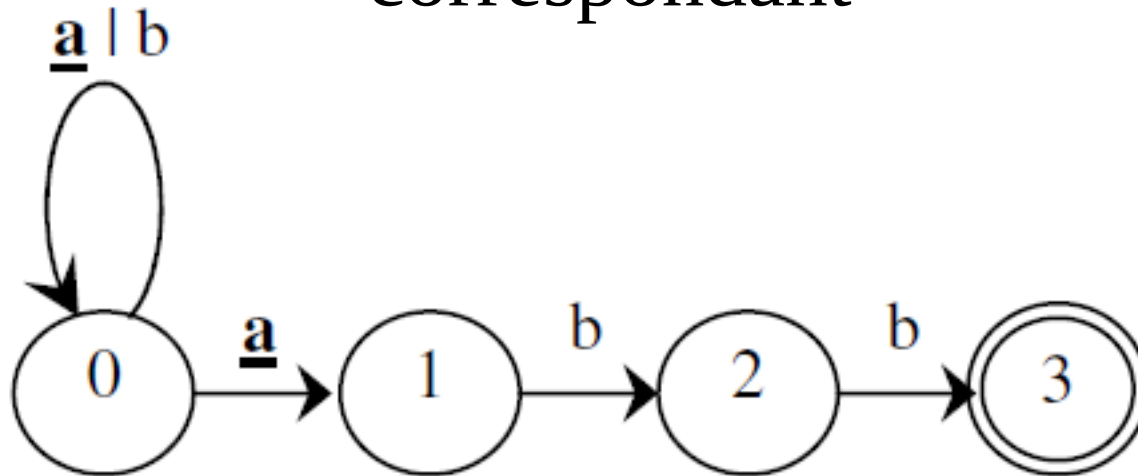


Exemple 2: a^*ba^*

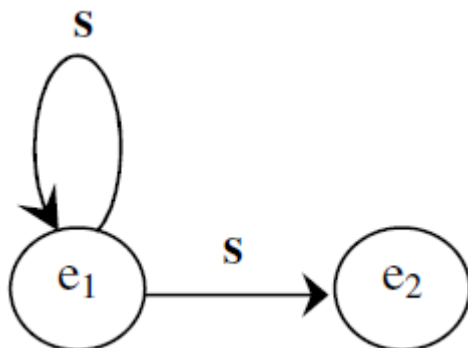


Exemple 3:

Soit l'ER suivante $(a|b)^*abb$, construire l'automate correspondant



Exemple 4: Ecrire le ER correspond a la l'automate suivant



$\Rightarrow S^*S$

Exercices

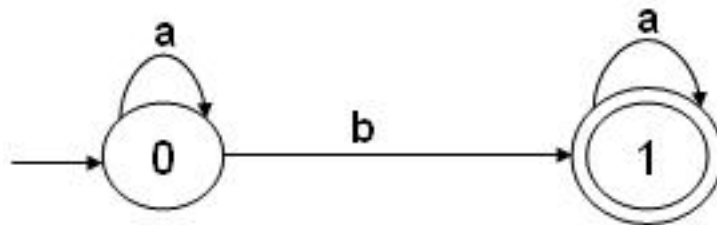
AFD peut être représentée à l'aide d'un tableau bidimensionnel appelé **matrice de transition**.

- Les lignes de cette matrice sont indexées par les états et les colonnes par les symboles de l'entrée.
Une case (q, a).

La matrice de transition de l'AFD de l'exemple est la suivante :

δ	a	b
0	0	1
1	1	

a^*ba^* .



Etant à l'état 0, on lit le symbole a et on passe à l'état 0.

Les **automates finis non-deterministes (AFND)** constituent un outil formel efficace pour transformer des expressions régulières en programmes efficaces de reconnaissance de langages.

Un AFN peut être représentée graphiquement comme un graphe orienté étiqueté, appelé graphe de transition, dans lequel les nœuds sont les états et les arcs étiquetés représentent la fonction de transition.

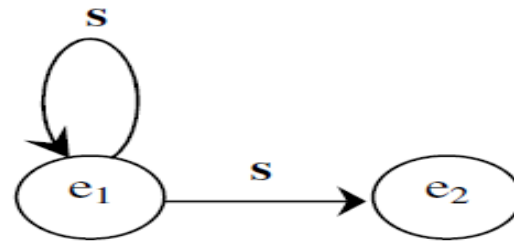
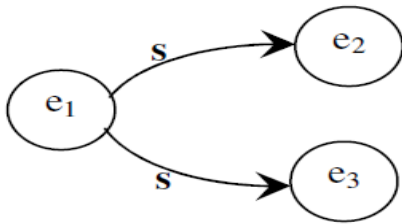
Par leur nature **non-deterministe**, ils sont plus compliqués que les **AFD**.

- La nature **non-deterministe** d'un **AFND** provient des points suivants :
- Il peut posséder plusieurs états initiaux.
- Il peut y avoir plusieurs transitions à partir d'un même état avec le même symbole de l'entrée.
- Il peut y avoir des **transitions spontanées** (c'est-à-dire sans lire aucun symbole d'entrée) d'un état vers un autre.

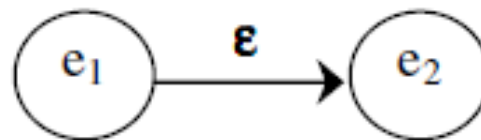
Automates Finis Non déterministes (AFN)

Un AFN est un automate qui présente l'une des deux situations suivantes :

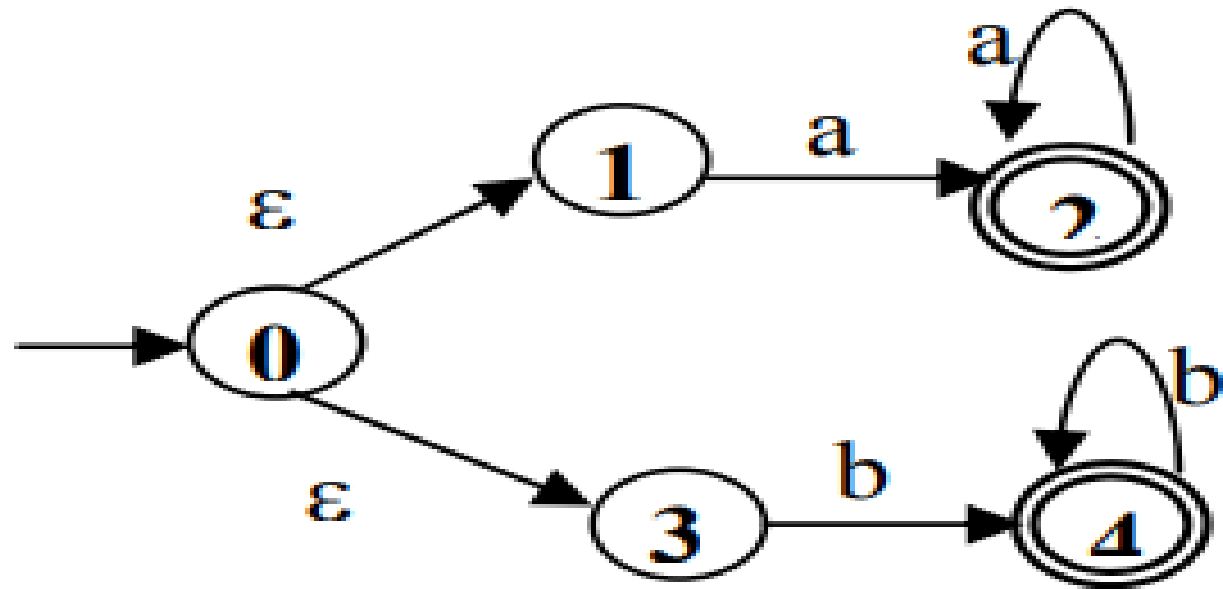
-A partir d'un même état de départ, il existe deux arcs sortant sur le même symbole, partant vers deux états différents:



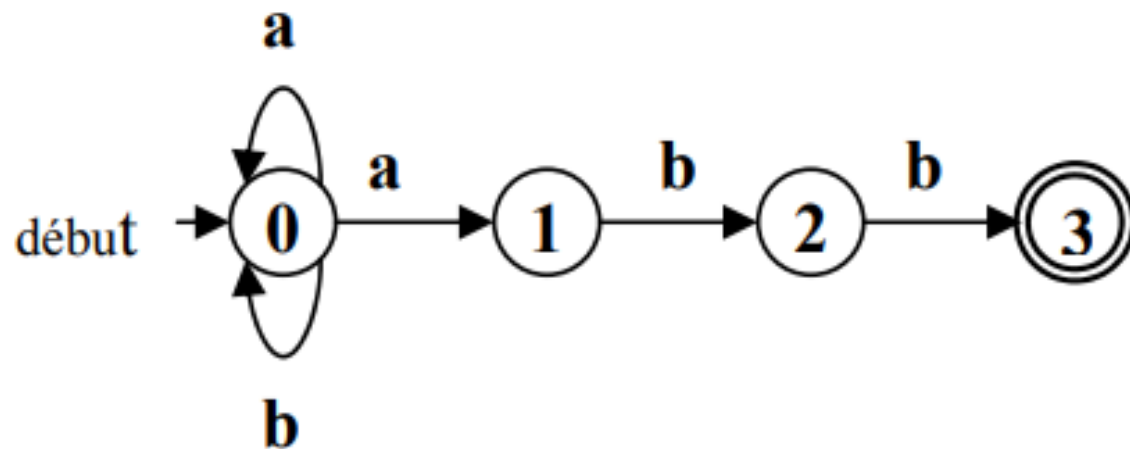
-Il existe une transition sur le symbole ϵ , appelée ϵ -transition. Celle-ci signifie un changement d'état sans consommation de symbole. Ce qui est un non déterminisme : faut-il ou non changer d'état quand rien ne se passe en entrée.



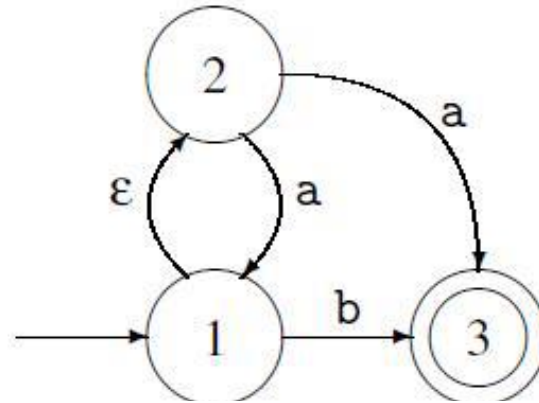
Exemple 1:



Un automate fini non déterministe



La figure suivante montre un exemple d'un **AFND** :



- $E = \{1, 2, 3\}$.
- $A = \{a, b\}$.
- $Q_0 = \{1\}$.
- **La matrice de transition :**

δ	a	b	ε
1		$\{3\}$	$\{2\}$
2	$\{1, 3\}$		
3			

- $F = \{3\}$.

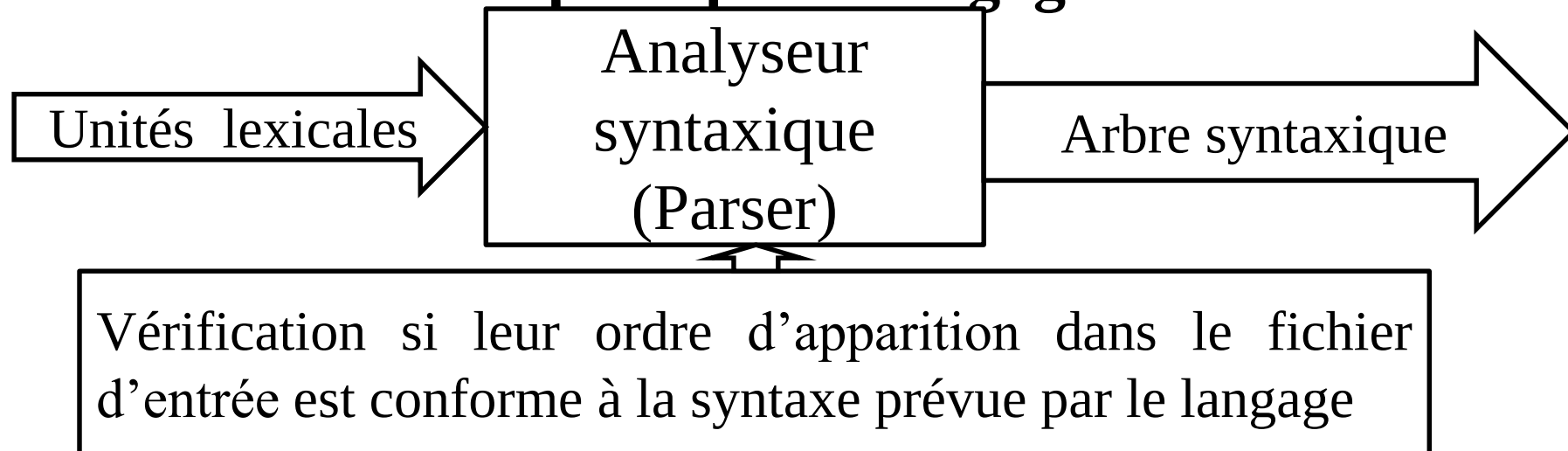
- On part de l'état initial 1. On lit le symbole ε et on passe à l'état 2.
- On lit le symbole a et on passe à l'état 1.

- Les automates finis déterministes produisent des connaisseurs plus rapides que les automates finis non déterministes.
- Un automate fini déterministe peut être beaucoup plus volumineux qu'un automate fini non déterministe.

Chapitre 3. Analyse Syntaxique

L'analyseur syntaxique(Parser) est la 2^{ème} phase de la compilation. La syntaxe doit donc être définie d'une manière régulière et sans ambiguïté. Ceci nécessite une spécification de la syntaxe réalisée à l'aide d'un langage de spécification syntaxique simple et universelle. Il s'agit de la notation **des grammaires**.

L'objectif de cette phase est **de vérifier si les éléments** qui proviennent de l'**analyseur lexical forment une chaîne d'unités lexicales acceptées par le langage**.



Grammaires

Une grammaire est définie sous forme d'une liste de productions permettant de couvrir toutes les possibilités syntaxiques du langage. Elle est définie régulièrement comme un quadruplet : $G(\mathbf{VT}, \mathbf{VN}, \mathbf{P}, \mathbf{S})$

- VT : Vocabulaire Terminal
- VN : Vocabulaire Non terminal
- P : un tableau de productions
- S : appelé Axiome de la grammaire, est le premier non terminal définissant la première production de la grammaire.

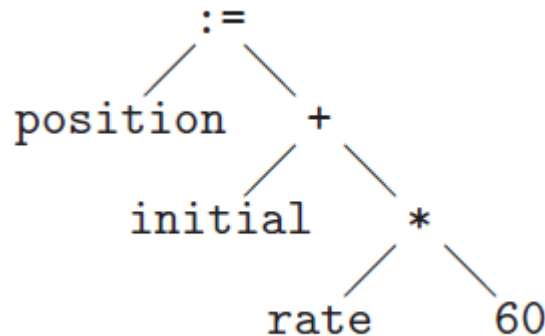
Le rôle principal de l'analyseur syntaxique est de trouver la structure du programme: c à d construire une représentation interne au compilateur et facilement manipulable. **Le parser construit l'arbre syntaxique (Parse tree) correspondant au code.**

Arbre syntaxique

Un arbre syntaxique (ou arbre de syntaxe, parse tree) est le schéma des dérivations nécessaires pour obtenir une sentence depuis l'axiome. **L'axiome va définir la racine de l'arbre.** Les feuilles de l'arbre sont les constituants de la sentence.

Exemple : Pour le code `:position := initial + rate * 60`

L'arbre syntaxique sera le suivant :

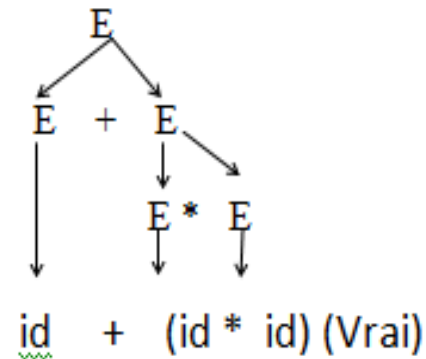
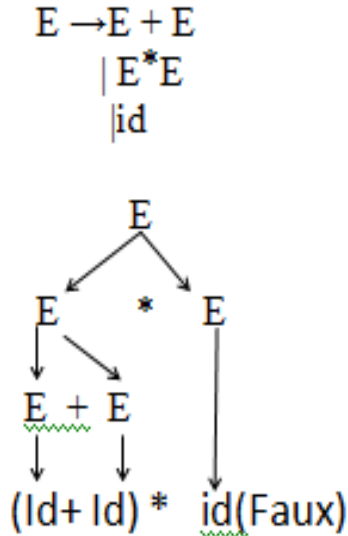


Complément : boucle, instructions conditionnelles

Grammaire ambiguë

Une grammaire est dite ambiguë si à une même sentence correspond deux ou plusieurs arbres syntaxiques et réponses différentes.

Exemple :



$VN\{E\}$

$VT\{+,*,id\}$

Analyseur prédictif descendant (LL):

Left to right scanning, left most derivation)

L'analyse prédictive s'appuie sur la connaissance des **premiers symboles** qui peuvent être engendrés par les parties droites des productions.

- Définir les ensembles **des annulables(null)**, **des premiers (first)** et **les suivants (Follow)**.
- Construire une table de décision appelée **Table d'Analyse** dans laquelle sont répertoriées les décisions de dérivation possibles.

Le prédicat Nullable:

Le null de x est vrai ssi, a partir de x il y a une dérivation vers ϵ

Null(x) est vrai ssi $x \longrightarrow^+ \epsilon$

Formellement : $\text{Null}(G) = \{X \in V \mid \text{Nullable}(X) = \text{Vrai}\} = \{X \in V \mid X \Rightarrow^* \epsilon\}$

Exemple 1: soient les grammaires suivantes :

$A \rightarrow \text{bid};$

$B \rightarrow a/B/\varepsilon$

A	B
0	1

Exemple 2: soit la grammaire suivante

$S \rightarrow ABC$
$A \rightarrow a/D$
$B \rightarrow b/\varepsilon$
$C \rightarrow c/\varepsilon$
$D \rightarrow \varepsilon$

S	A	B	C	D
0	0	1	1	1
0	1	1	1	1
1	1	1	1	1
1	1	1	1	1

|

La fonction Premier (First)

Définition Soit $\alpha \in (V \cup T)^*$ une forme d'une $G = (VN, VT, P, S)$. On définit l'ensemble $\text{Premier}(\alpha)$ (ou $\text{First}(\alpha)$), comme étant l'ensemble des **terminaux** qui peuvent apparaître au début d'une forme dérivable à partir de α .

$$\text{Premier}(\alpha) = \{ a \in T \mid \alpha \Rightarrow^* a\beta, \beta \in (V \cup T)^* \}$$

Les règles suivantes permettent de calculer cette fonction :

1. $\text{Premier}(\varepsilon) = \emptyset$.
2. $\text{Premier}(a) = \{a\}, \forall a \in T$.
3. Si $\text{Nullable}(\alpha) = \text{Vrai}$, $\text{Premier}(\alpha\beta) = \text{Premier}(\alpha) \cup \text{Premier}(\beta)$.
4. Si $\text{Nullable}(\alpha) = \text{Faux}$, $\text{Premier}(\alpha\beta) = \text{Premier}(\alpha)$.
5. $\text{Premier}(X) = \text{Premier}(\alpha_1) \cup \dots \cup \text{Premier}(\alpha_n), \forall X \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$.

Exemple de des premiers(first)

Exemple:



$E \rightarrow TE'$
$E' \rightarrow + TE'$
$\quad \quad \quad / \varepsilon$
$T \rightarrow FT'$
$T' \rightarrow * FT'$
$\quad \quad \quad / \varepsilon$
$F \rightarrow (E)$
$\quad \quad \quad / \text{int}$

Les nullables (Null)

E	E'	T	T'	F
0	1	0	1	0

Les premiers (First)

E	E'	T	T'	F
.	.	.	.	.
0	{+}	0	{*}	{(, <u>int</u> }
0	{+}	{(, <u>int</u> }	{*}	{(, <u>int</u> }
{(, <u>int</u> }	{+}	{(, <u>int</u> }	{*}	{(, <u>int</u> }
{(, <u>int</u> }	{+}	{(, <u>int</u> }	{*}	{(, <u>int</u> }

La fonction Suivant (Follow)

On définit l'ensemble $\text{Suivant}(X)$ (ou $\text{Follow}(X)$) pour tout symbole non-terminal $X \in V$, comme étant l'ensemble des terminaux qui peuvent apparaître juste après le symbole X dans une dérivation à partir de l'axiome S de la grammaire G .
Formellement :

$$\text{Suivant}(X) = \{\alpha \in T \mid S \Rightarrow^* \alpha X \alpha \beta \text{ avec } \alpha, \beta \in (V \cup T)^*\}$$

Exemple: soit la grammaire suivante

Exemple1



$S \rightarrow E \text{ eof}$
 $E \rightarrow TE'$
 $E' \rightarrow +TE' / -TE' / \varepsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' / TE' / \varepsilon$
 $F \rightarrow (E) / id$



S	E	E'	T	T'	F
0	0	0	0	0	0
{ <u>eof</u> }	{ <u>eof.</u> }	0	{+, -}	0	{*, /}
{ <u>eof</u> }	{ <u>eof.</u> }	{ <u>eof.</u> }	{+, -, <u>eof.</u> }	{+, -}	{*, /, +, -}
{ <u>eof</u> }	{ <u>eof.</u> }	{ <u>eof.</u> }	{+, -, <u>eof.</u> }	{+, -, <u>eof</u> }	{*, /, +, -, <u>eof</u> }
{ <u>eof</u> }	{ <u>eof.</u> }	{ <u>eof.</u> }	{+, -, <u>eof.</u> }	{+, -, <u>eof</u> }	{*, /, +, -, <u>eof</u> }



EXEMPLE: Soit la grammaire suivante

$$\begin{array}{lll} E & \rightarrow & T E' \\ E' & \rightarrow & + \mid T E' \\ & & \mid \epsilon \\ T & \rightarrow & F T' \\ T' & \rightarrow & * F T' \\ & & \mid \epsilon \\ F & \rightarrow & (E) \\ & & \mid \text{int} \end{array}$$

Soit la grammaire suivante

$$T \rightarrow E T' | \varepsilon$$

$$T' \rightarrow * E T'$$

$$E \rightarrow F E'$$

$$E' \rightarrow + F E' | \varepsilon$$

$$F \rightarrow (T) | \text{int}$$

Donnez la table des annulable, premiers (First) et des suivants (Follow)

Construction de la table d'analyseur syntaxique LL(1) (parsing table)

A l'aide des premiers et des suivants, on construit la table d'analyseur syntaxique($X; a$) de la manière suivante pour chaque production $X \rightarrow \lambda$,

- on pose $T(X; a) = \lambda$ pour tout $a \in \text{first}(\lambda)$
- si $\text{null}(\lambda)$, on pose aussi $T(X; a) = \lambda$ pour tout $a \in \text{follow}(X)$

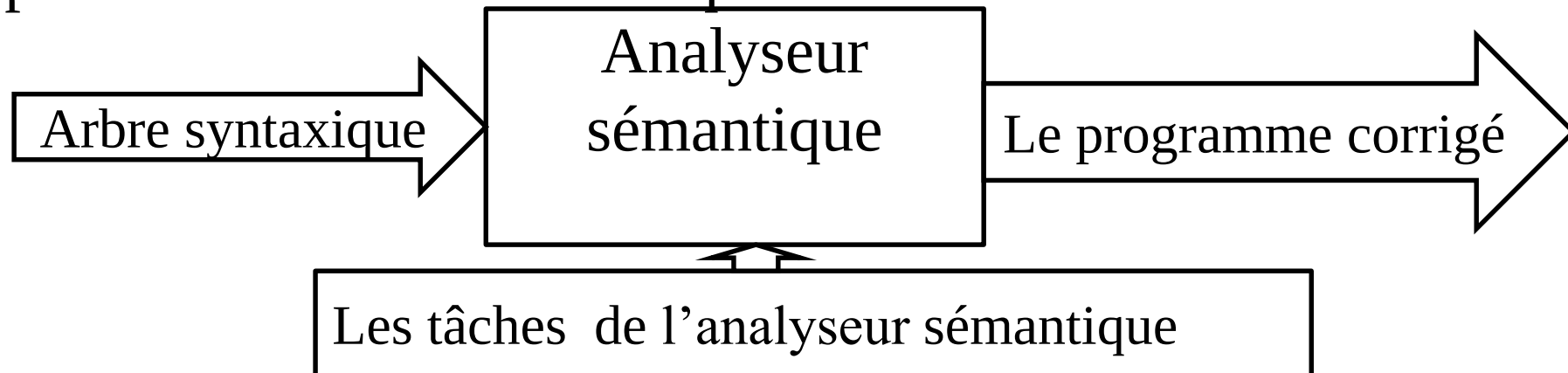
	+	*	(	)	int	#
E			TE'		TE'	
E'	$+TE'$			ϵ		ϵ
T			FT'		FT'	
T'	ϵ	$*FT'$		ϵ		ϵ
F			(E)		int	

Chapitre 4. Analyse sémantique

Après l'analyse lexicale et l'analyse syntaxique, la phase suivante dans la compilation est l'analyse sémantique.

Certaines expressions peuvent être syntaxiquement correctes mais ne pas avoir de sens par rapport à la sémantique du langage.

L'analyse sémantique étudie l'arbre de syntaxe abstraite produit par l'analyse syntaxique pour éliminer au maximum les programmes qui ne sont pas corrects du point de vue de la sémantique.



Quelques de tâches de l'analyseur sémantique

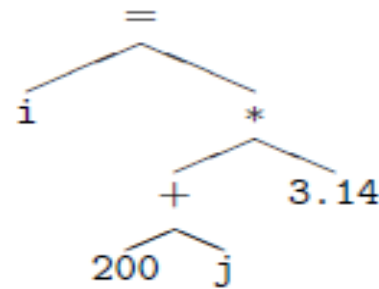
Des exemples de tâches liées au contrôle de type sont :

- construire et mémoriser des représentations des types définis par l'utilisateur, lorsque le langage le permet ;
- traiter les déclarations de variables et fonctions et mémoriser les types qui leur sont appliqués ;
- vérifier que toute variable référencée et toute fonction appelée ont bien été préalablement déclarées ;
- vérifier que les paramètres des fonctions ont les types requis ;
- contrôler les types des opérandes des opérations arithmétiques et en déduire le type du résultat ;
- Etc.

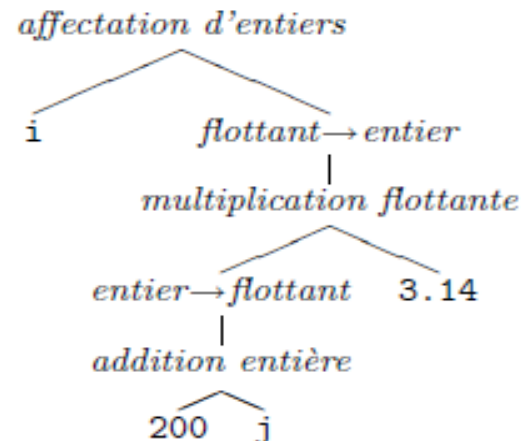
Exemple de construction d'un arbre abstrait

Le travail sémantique à faire consiste à « remonter les types », depuis les feuilles vers la racine, rendant concrets les opérateurs et donnant un type précis aux sous-arbres.

Soit l'expression suivante:



le contrôle de type transforme l'arbre précédent en quelque chose qui ressemble à ceci,



Chapitre 5 Génération du code intermédiaire

5.1 Objectif

Produire à partir du code source un code intermédiaire indépendant de toute exigence matérielle.

La phase de gestion du code finale sera indépendante des particularités du code source.

Facilité de changement du code final sans avoir à toucher la phase d'analyse

5.2 Réalisation

- * Code à 3 addresses :

suite d'instructions faisant intervenir au maximum 3 adresses:

Résultat := Opérande1 Opération Opérande2.

Généralisation de cette forme pour englober tout type de construction du langage.

Exemple: $a=b+c$

Example

3-address code

1	t1 = b*c
2	t2 = a+t1
3	t3 = b*c
4	t4 = d/t3
5	t5 = t2-t4

Quadruples

op	arg ₁	arg ₂	result
*	b	c	t1
+	a	t1	t2
*	b	c	t3
/	d	t3	t4
-	t2	t4	t5

Triples

	op	arg ₁	arg ₂
0	*	b	c
1	+	a	(0)
2	*	b	c
3	/	d	(2)
4	-	(1)	(3)

Optimisation de code et génération du code cible

. Une optimisation ne doit jamais modifier le code de façon à ce que l'exécution du programme cible ne soit plus compatible avec l'exécution du programme source. La phase d'optimisation de code intermédiaire consiste surtout à :

- Eliminer des expressions et données redondantes
- Optimiser de point de vue temps d'exécution
- Supprimer les calculs invariants dans les boucles
- Augmenter le temps de compilation
- Optimiser les boucles.
- Eliminer les codes morts et variables mortes

Exemple

Exemple 1: Elimination des expressions redondantes

$a = b + c$

$b = a - d$

$c = b + c$

$d = a - d$

$\Rightarrow$

$a = b + c$

$b = a - d$

$c = b + c$

$d = b$

Exemple2

$t6 = 4 * i$

$x = a[t6]$

$t7 = 4 * i$

$t8 = 4 * j$

$t9 = a[t8]$ après élimination on aura:

$a[t7] = t9$

$t10 = 4 * j$

$a[t10] = x$

$t6 = 4 * i$

$x = a[t6]$

$t8 = 4 * j$

$t9 = a[t8]$

$a[t6] = t9$

$a[t8] = x$

Génération du code cible

La génération du code final traduit le langage intermédiaire en un langage machine dépendant de l'architecture (registres et mémoire, adresses, pile d'exécution, compilation séparée et édition de liens, etc.). La traduction des instructions abstraites en instructions machine dépend aussi du jeu d'instruction de l'architecture cible.

Exemple:

MOVF R2,id3

MULF #60.0, R2

MOVF R1,id2

ADDF R2, R1

MOVF R1, id1

Les outils de l'analyse lexical et syntaxique

Lex: Générateur d'analyse lexical.

- Produit un automate fini déterministe minimal permettant de reconnaître les unités lexicales.
- L'automate est produit sous la forme d'un programme C.
- Un programme en lex définit un ensemble de schémas qui sont appliqués à un flux textuel.
- Chaque schéma est représenté sous la forme d'une expression régulière.
- Lorsque l'application d'un schéma réussit, les actions qui lui sont associées sont exécutées.
- Il existe plusieurs versions de lex (flex,..)

yacc (Yet Another Compiler Compiler):
Générateur d'analyse syntaxique.

- Produit un analyseur syntaxique pour le schéma de traduction.
- L'analyseur est produit sous la forme d'un programme C.
- Il existe plusieurs versions de yacc Bison.

Communication entre Lex et YACC

Entre les analyseurs syntaxique et lexical :

- Le fichier `lex.yy.c` définit la fonction `yylex()` qui analyse le contenu du flux `yyin` jusqu'à détecter une unité lexicale. Elle renvoie alors le code de l'unité reconnue.
- L'analyseur syntaxique (`yyparse()`) appelle la fonction `yylex()` pour connaître la prochaine unité lexicale à traiter.
- Le fichier `calc.tab.h` associe un entier à tout symbole terminal. `#define NOMBRE 258`

Compilateur GCC

GCC est une suite de compilateurs distribuée par le projet GNU.

Gcc est l'acronyme de GNU (Gnu's Not Unix) Compilers Collection;

Il s'agit en fait d'une collection qui regroupe des compilateurs supportant de nombreux langages, tels que le C, le C++, et autres.

Il fournit en outre les différentes bibliothèques standard pour ces langages.

MinGW est l'acronyme de **Minimalist Gcc** for **Windows**

Le format d'exécutables sous Windows.

Cygwin est un **émulateur** Linux qui fournit un compilateur croisé pour Windows et nécessite donc une orientation plus linuxienne.

- gcc : le compilateur C de GNU ;
- g++ : le compilateur C++ de GNU ;
- gcj : le compilateur Java de GNU
- gnat : le compilateur Ada de GNU ;
- gfortran : le compilateur Fortran 95 de GNU

References bibliographiques

1. Danny Dubé; Compilation et interprétation, Université Laval ; Révision automne 2013 ;
2. Noureddine Chenfour; Principes et Techniques de Compilation ; Université Sidi Mohamed Ben Abdellah Faculté des Sciences Dhar El-Mehraz Fès ; 2010 ;
3. Guillaume Burel; Cours de compilation ; École nationale supérieure
4. d'informatique pour l'industrie et l'entreprise ; 13 décembre 2015
5. Tanguy Risset; Cours de Compilation, Master 1, 2004-2005 ;
6. Damien Simon ; Laboratoire de probabilités et modèles aléatoires, Université Pierre et Marie Curie Année universitaire 2016–2017